LUISS REPLY

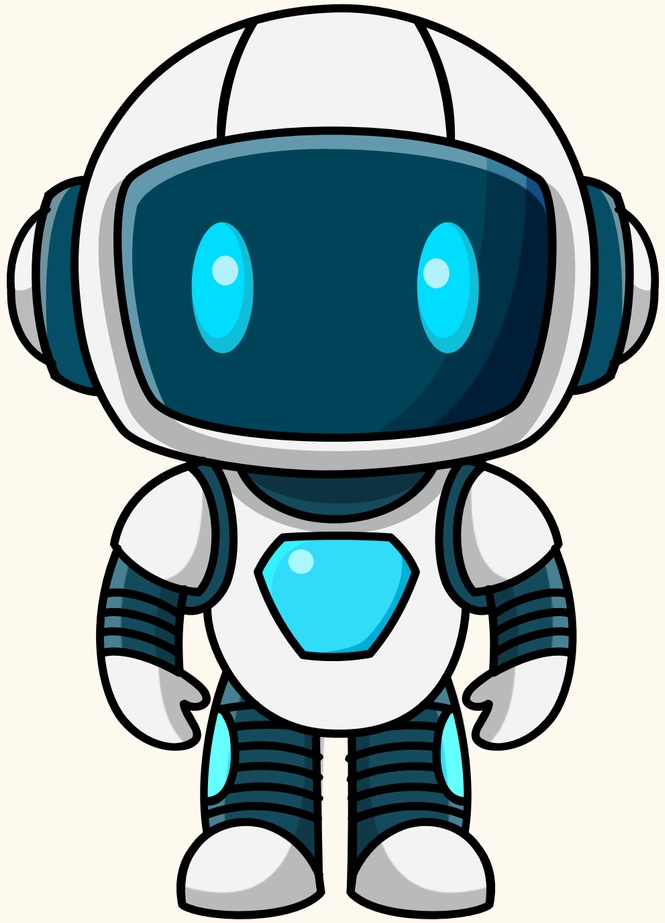
AGENTS4DQ - Agents for Data Quality

A Multi-Agent System for automated data quality assessment

Group 5

Agenda

- **The Problem:** Data Quality in NoiPA
- **The Solution:** AGENTS4DQ
- **System Architecture**
- **Agents in Detail**
- **Key Design Choices**
- **Experiment 1:** *spesa.csv*
- **Experiment 2:** *attivazioniCessazioni.csv*
- **Key Results**
- **Streamlit Demo**
- **Conclusions & Future Work**



The Problem

The Context: NoiPA & Data Quality

1. NoiPA: digital platform of the **Italian Ministry of Economy and Finance**

2. **Manages salaries, timesheets, and tax/social security obligations** for the Public Administration

3. Receives datasets from heterogeneous sources (**CSV, JSON, databases**)

4. Currently: **validation is either manual or completely absent**

Risks:

- Decisions based on unreliable data
- Cascading errors in downstream processes
- High cost of manual correction

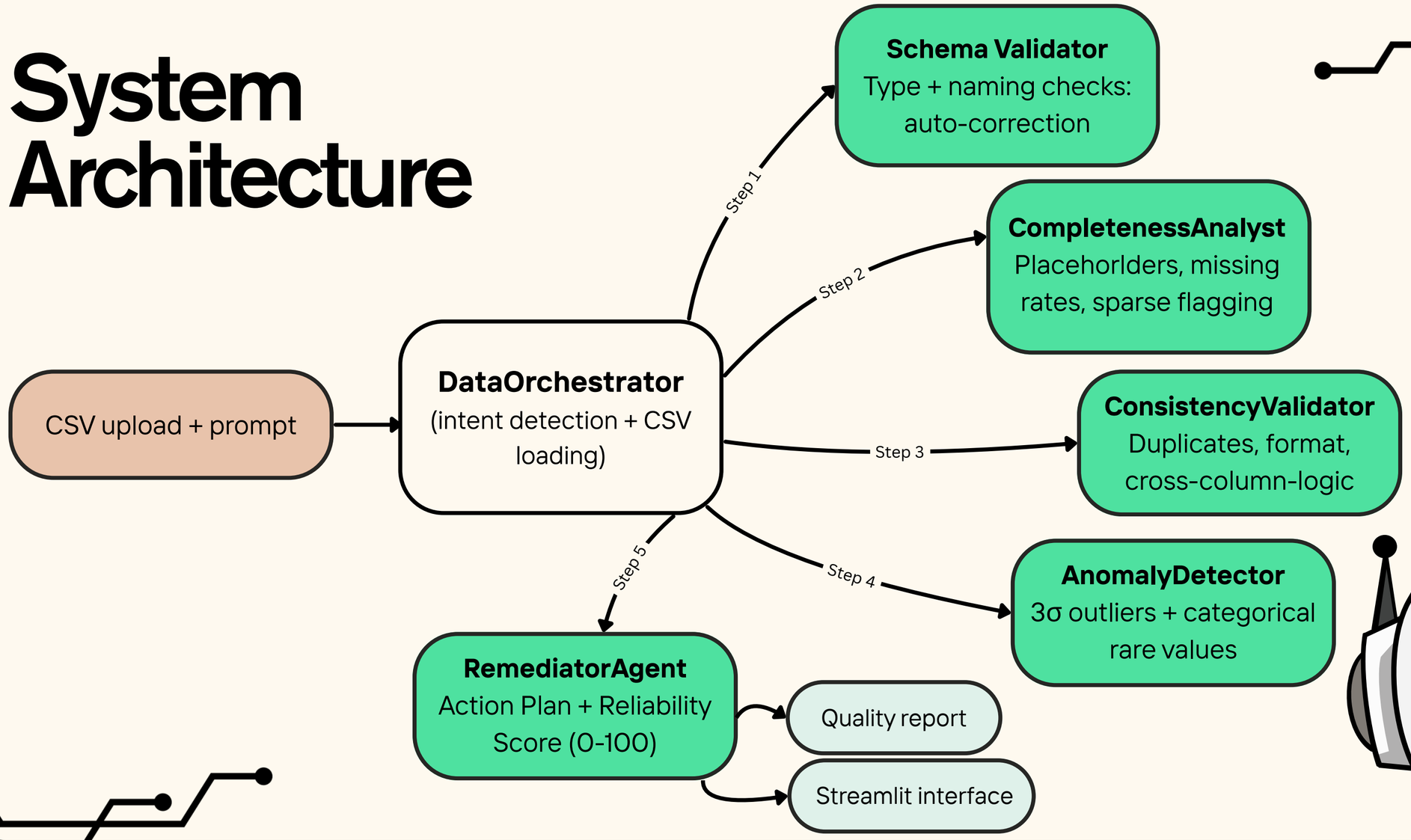
The Solution

A multi-agent system that automatically ingests a raw CSV dataset and:

1. **Validates & corrects** the schema (data types and naming conventions)
2. **Analyzes completeness** (nulls, placeholders, sparse columns/rows)
3. **Checks consistency** (duplicates, formats, cross-column logical rules)
4. **Detects anomalies** (statistical outliers and rare categorical values)
5. **Generates a report** with a Data Reliability Score (0–100) and an Action Plan

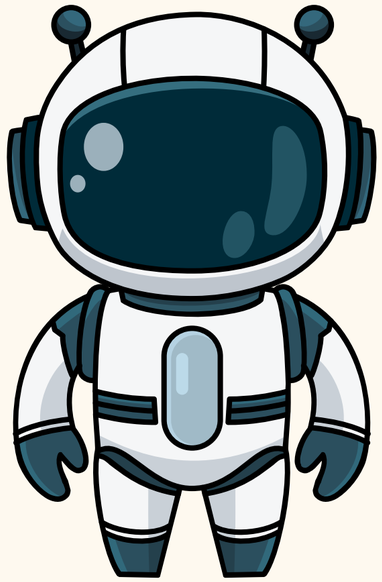
Final output: Structured quality report + Streamlit web interface

System Architecture



Agent Detail: **DataOrchestrator**

Field	Detail
LLM	Gemma 4 31B IT
Input	User text + CSV/XLSX file
Role	Parses user intent and loads the dataset
Output	Pandas DataFrame + status message



Note: Entry point of the pipeline: determines whether the user actually wants to analyze the file.

Agent Detail: **SchemaValidator**

Field	Detail
LLM	Gemini 3.1 Flash Lite Preview
4 phases	Type check → Type correction → Naming check → Naming correction

How does it works:

- **Compares dominant regex patterns against current pandas dtypes**
- **Example: column with pattern *NN.NNN* but dtype *object* → flag mismatch**
- **Generates a JSON correction map *{column: target_type}***
- **Automatically fixes types and names with deduplication rules**

Example issues detected

- *ente%code* → special character %
- *2cod_imposta* → leading digit
- *SPESA TOTALE* → inconsistent casing

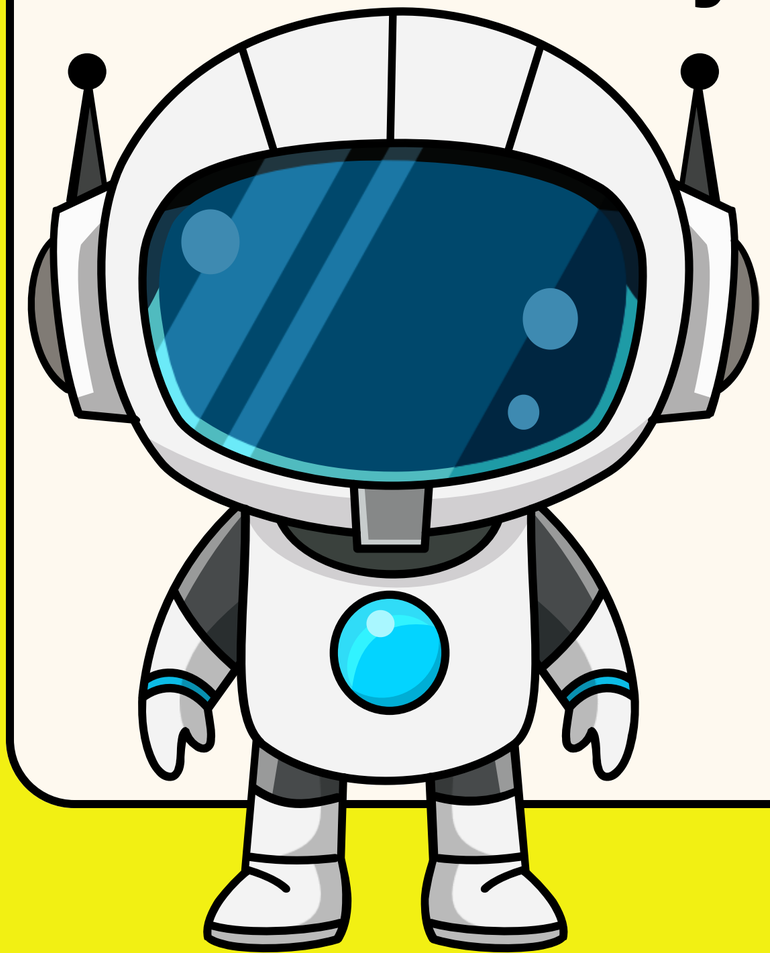
Agent Detail: **CompletenessAnalyst**

Field	Detail
LLM	Gemma 4 31B IT
Strength	Italian language understanding for placeholder detection

4 steps pipeline

1. **Placeholder Detection via LLM**: identifies strings like *"-"*, *"null"*, *"N/A"*, *"sconosciuto"* → replaces with *pd.NA*
2. **Missing % per column**: columns with >50% missing are candidates for removal
3. **Missing % per row**: rows with >50% missing are candidates for removal
4. **LLM Summarisation**: human-readable report

Agent Detail: **ConsistencyValidator**



Field	Detail
LLM	Gemini 3.1 Flash Lite Preview
3 checks	Duplicates · Format · Cross-column logic

Check details:

- **Duplicates**: exact (auto-removed) + key-column based (e.g. *_id*, *codice_fiscale*)
- **Format**: regex pattern fingerprinting - e.g. {*"NN/NN/NNNN": 990, "NN-NN-NNNN": 10*} reveals inconsistency
- **Cross-column logic**: on a sample of 30 rows, LLM infers logical rules (e.g. *tipo_imposta* ≡ *Tipo Imposta* = redundant mirror columns)

Agent Detail: **AnomalyDetector**

Field	Detail
LLM	Gemma 4 31B IT
2 detection types	Univariate (numerical) · Categorical

Categorical anomaly detection:

- LLM selects suitable categorical columns
- Values appearing in <1% of rows → flagged as rare/anomalous

Univariate outlier detection:

- LLM selects semantically relevant numerical columns
- 3σ rule: values beyond 3 standard deviations from the mean → flagged
- If >10 outliers, details written to a separate report

Agent Detail: **RemediatorAgent**

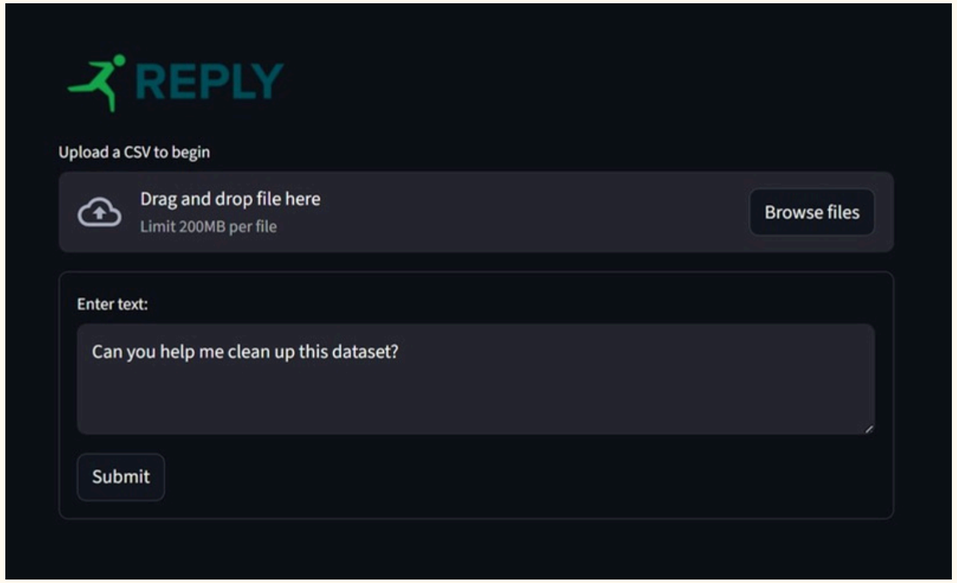
Field	Detail
LLM	Gemma 4 31B IT
Output	Data Reliability Score (0–100) + Human-in-the-Loop Action Plan

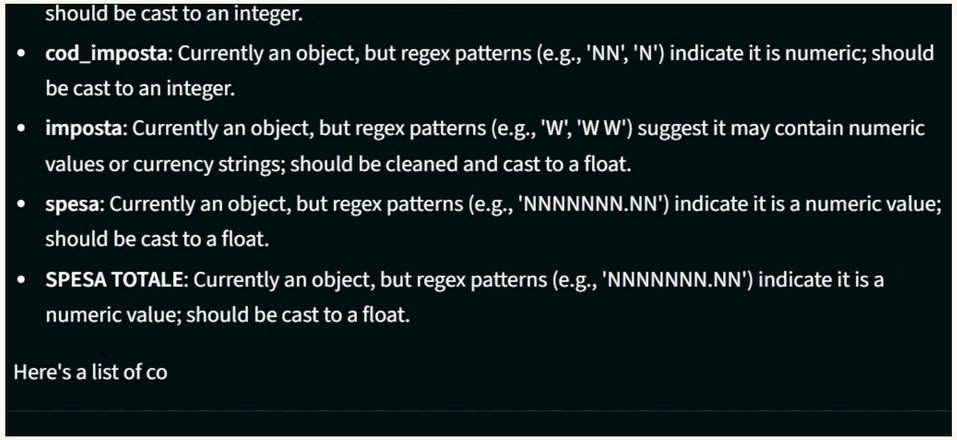
The final report includes:

- Global score with justification
- Imputation strategies (median for numerical, mode for categorical)
- Outlier handling recommendations (cap, delete, investigate)
- Instructions to resolve cross-column inconsistencies

Philosophy: bridging the gap between automated detection and human decision-making

Demo Interface





Key points:

- Interactive CSV upload
- Step-by-step streaming output with typewriter effect
- Final report viewable directly in the interface

Design Choices

- SQLite caching to avoid redundant API calls
- Streamlit frontend with typewriter-style streaming output

Key Design Choices

1

LLM-Driven Column Selection

No hard-coded rules, agents ask the LLM which columns are relevant for each check. The system is **dataset-agnostic**.

2

Auto-Correction + Human-in-the-Loop

High-confidence fixes applied automatically (type casting, naming, deduplication). Domain-dependent decisions → Action Plan for the human operator.

3

Regex Pattern Fingerprinting

Each cell → shape string (digits→N, letters→W, punctuation preserved). Enables format consistency detection without hard-coded formats.

4

Hybrid: Statistical + LLM

Statistical checks (3σ, frequency thresholds, exact duplicates) combined with LLM reasoning (placeholder detection, cross-column logic, naming conventions).

Experiment 1
spesa.csv

Baseline:
manual inspection in a spreadsheet editor

Dataset: 7,543 rows × 18 columns (NoiPA expenditure records)

Intentional noise introduced:

- Naming violations: *ente%code*, *2cod_imposta*, *cod imposta ext*, *Tipo Imposta*
- High missing %: *note*, *fonte_dato*
- Mixed types: numerical columns stored as strings
- Redundant columns: *spesa = SPESA TOTALE*, *cod_imposta = 2cod_imposta = cod imposta ext*

Experiment 2

attivazioniCessazioni.csv

Purpose:
test generalization to a structurally different dataset

Dataset: 20,102 rows × 19 columns (personnel activations/terminations)

Intentional noise

- *3descrizione* → leading digit
- *regione%sede* → special character
- *att ivazioni* → space in column name
- Province casing inconsistencies: Aq vs AQ
- Redundant mirror columns

Key Results

- Schema Validation Type **mismatches and naming violations correctly identified and auto-corrected** on both datasets.
- Consistency Pattern fingerprinting detects **format anomalies**. **Exact duplicates removed**. **Redundant/mirror columns identified**.
- Completeness **placeholder detection effective**. Columns like note and fonte_dato flagged (>50% missing).
- Anomaly Detection 3σ **flags extreme values** in spesa. **Rare entity descriptions and tax types identified**.

Main Takeways !

1. Multi-agent + LLM = data quality automation without **hard-coded rules**

2. Combining statistical methods with LLM reasoning → adaptability to different schemas

3. Goes beyond detection: applies high-confidence automatic corrections

4. Transparent delegation via Action Plan + Reliability Score

Limitations & Future Work

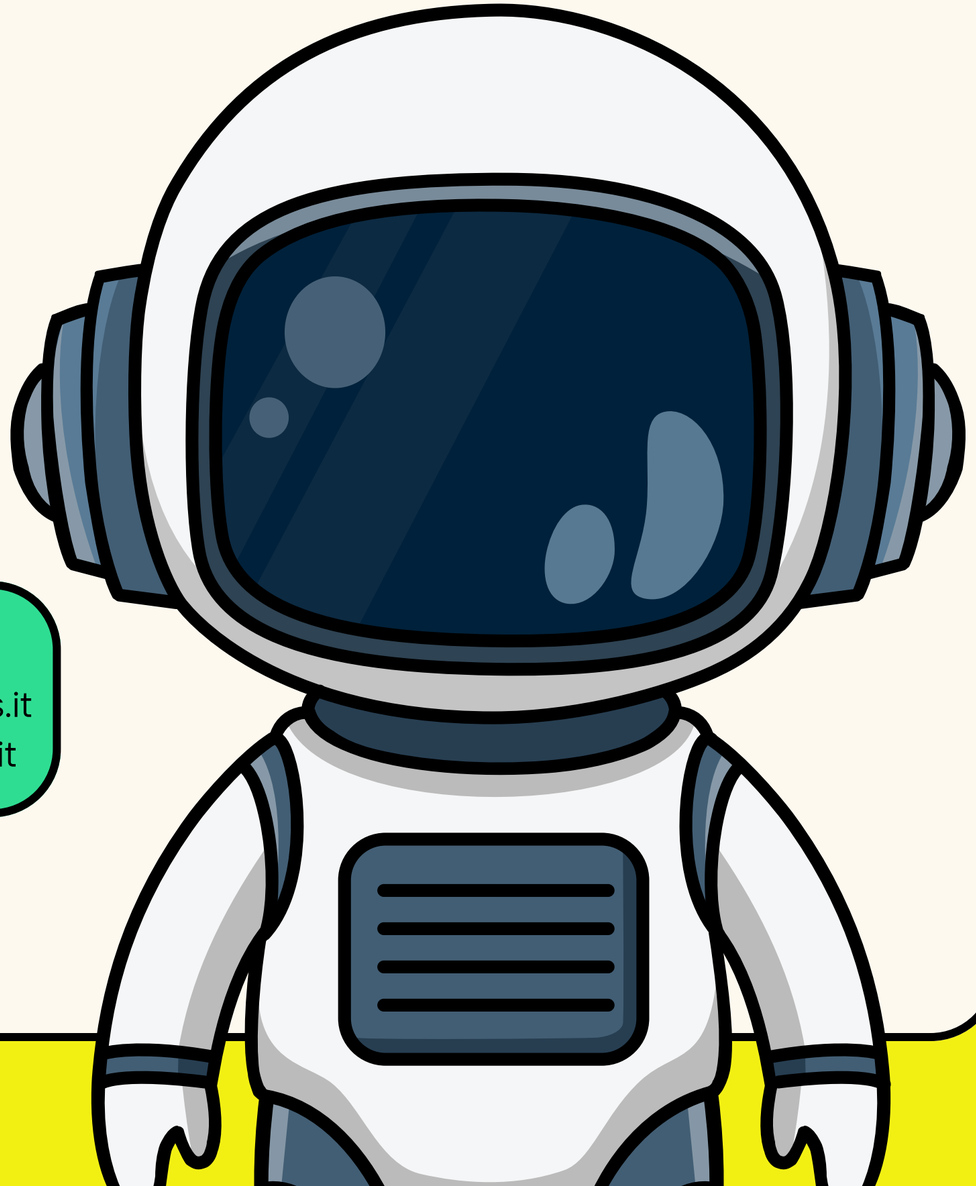
Limitation	Possible Evolution
Sequential pipeline	Parallel execution to reduce latency
Limited automatic fixes	Interactive approve/reject interface for each suggestion
CSV-only support	Extension to JSON and database connections
Only 3σ for outliers	Isolation Forest, DBSCAN for non-normal distributions

GROUP 5

THANK YOU!

Contact Info

Maria Dichio
Armando Fornario
Gianfranco Votta

maria.dichio@studenti.luiss.it
armando.fornario@studenti.luiss.it
gianfranco.votta@studenti.luiss.it

Repository:
https://github.com/gianfrancovotta/AGENTS4DQ-807861.git